

MINERÍA DE DATOS • UNIDAD 2 — DEL TEXTO AL SIGNIFICADO

Introducción al Procesamiento de Lenguaje Natural

*Del texto crudo al primer motor de búsqueda:
preprocesamiento, espacio vectorial y TF-IDF*

Ramsés Alejandro Camas Nájera

Universidad Politécnica de Chiapas • 9º cuatrimestre • Semana 5, Sesión 1 • Junio 2026

DÓNDE ESTAMOS

El curso responde una sola pregunta, tres veces

¿Cómo extraes conocimiento no obvio de datos que nadie te dijo que buscaras?

UNIDAD 1 • MAYO

El dato antes del modelo

Scraping, clustering, reglas de asociación y anomalías sobre datos tabulares.

COMPLETADA

UNIDAD 2 • JUNIO

Del texto al significado

IR, NLP tradicional, embeddings y búsqueda semántica con Qdrant.

← ESTAMOS AQUÍ

UNIDAD 3 • JUL-AGO

Minería en producción

RAG, transformers, privacidad, MLOps y proyecto integrador final.

PRÓXIMAMENTE

Pregunta central de la Unidad 2: *¿cómo encuentras patrones en datos no estructurados, y por qué la forma en que representas el texto cambia todo lo que puedes descubrir?*

Agenda — 3 horas, 3 bloques y un fallo en vivo



Bloque 1 — El texto como dato

Qué es NLP, por qué el texto es difícil e Information Retrieval como problema fundamental.

~45 min



Bloque 2 — Preprocesamiento

Tokenización, normalización, stopwords, stemming vs. lematización y regex. Demo en vivo.

~60 min



Bloque 3 — Del texto al vector

Espacio vectorial, Bag of Words, TF-IDF desde cero y similitud coseno. Mini buscador funcionando.

~60 min



Cierre — La falla demostrable

Una consulta que TF-IDF no puede resolver. El teaser de los embeddings.

~15 min

BLOQUE

01

El texto como dato

Todo lo que aprendieron en la Unidad 1 opera sobre vectores numéricos. ¿Y cuando el dato es lenguaje?

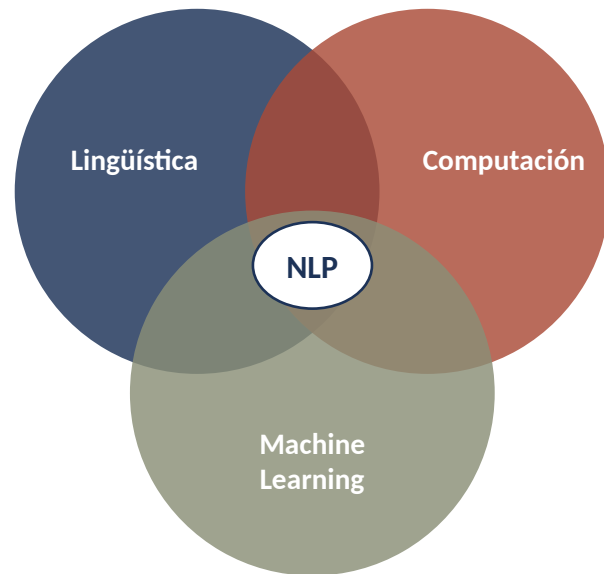
- Qué es NLP y dónde vive dentro de Data Mining
- Por qué el texto es genuinamente difícil
- Information Retrieval: 60 años de historia

¿Qué es el Procesamiento de Lenguaje Natural?

NLP es el conjunto de técnicas computacionales para analizar, entender y generar lenguaje humano — convertir texto no estructurado en estructuras que un algoritmo puede procesar.

Dentro de Data Mining:

- Misma filosofía de la Unidad 1: descubrir patrones sin hipótesis previa.
- Lo que cambia es el tipo de dato: texto en lugar de tablas.
- Text mining = NLP al servicio del descubrimiento de conocimiento (KDD).



El problema: sus algoritmos comen vectores

~80%

*de los datos que genera una
organización son no estructurados
— y la mayoría es texto.*

IDC / estimación de industria

Lo que ya dominan (Unidad 1)

K-Means, DBSCAN, Apriori, Isolation Forest... todos operan sobre puntos en $\mathbb{R}^n$: vectores numéricos con features definidas.

Lo que el mundo produce

Correos, tickets de soporte, reseñas, contratos, expedientes, posts, transcripciones. Nada de eso trae columnas ni features.

Por qué el texto es genuinamente difícil



Ambigüedad léxica

“Fui al banco” — ¿institución financiera o mueble para sentarse? La misma palabra, significados distintos.



Sinonimia

“rey” y “monarca” significan lo mismo, pero son cadenas de caracteres totalmente diferentes.



Contexto y sarcasmo

“Qué buen servicio, solo esperé dos horas” — la polaridad real es opuesta a las palabras usadas.



Variación y ruido

Errores ortográficos, abreviaturas, emojis, spanglish: “q oso, el wifi no sirve 🙄”.

Y el español lo complica más

La morfología del español es mucho más rica que la del inglés — y casi toda la literatura de NLP se diseñó para inglés.

Fenómeno	Inglés	Español
Conjugación verbal	run, runs, ran, running (4)	corro, corres, corre, corremos, corrían, correríamos... (50+ formas)
Género y número	the tall engineer	el ingeniero alto / la ingeniera alta / los ingenieros altos...
Diminutivos y afectivos	casi inexistentes	café → cafecito, cafecillo; ahora → ahorita (¡que ni significa “ahora”!)
Acentos y diéresis	no existen	papa ≠ papá; el ≠ él; pingüino — y la gente los omite al escribir
Variación regional	moderada	popote/pajilla/sorbete; chamba, platicar, “qué onda”

Consecuencia: las decisiones de preprocesamiento que tomemos hoy importan más en español que en inglés.

Information Retrieval: el problema fundamental

Dado un corpus de documentos y una consulta del usuario, **¿cómo encuentras los documentos más relevantes?** Es el problema detrás de Google, Elasticsearch, tu buscador de correos y el retrieval de un RAG.



60 años de historia que ningún curso de algoritmos les enseñó — y que hoy recorreremos en 3 horas hasta 1972.

El baseline histórico: Boolean retrieval

Cada documento es un conjunto de palabras. Las consultas combinan términos con AND, OR y NOT sobre una matriz de incidencia término–documento.

Corpus de juguete (lo usaremos toda la clase)

d₁: “La lluvia provocó inundaciones en Tuxtla”

d₂: “La sequía afecta cultivos de maíz en Chiapas”

d₃: “El café de Chiapas rompe récord de exportación”

Consulta: chiapas AND sequía → d₂

término	d ₁	d ₂	d ₃
lluvia	1	0	0
sequía	0	1	0
chiapas	0	1	1
café	0	0	1
tuxtla	1	0	0

Matriz de incidencia: 1 si el término aparece en el documento.

Por qué Boolean no basta



Todo o nada

Un documento cumple la consulta o no. No hay “casi relevante”: lluvia AND chiapas descarta a d_1 por una sola palabra.



Sin ranking

Si 10,000 documentos cumplen, ¿cuál muestras primero? Boolean no tiene noción de orden por relevancia.



Feast or famine

Las consultas devuelven demasiado (OR) o demasiado poco (AND). El usuario termina programando lógica en vez de buscar.



Todas las palabras pesan igual

“de” cuenta lo mismo que “sequía”. La intuición dice que unas palabras informan más que otras — Boolean no puede expresarlo.

BLOQUE

02

Preprocesamiento

Cada técnica es la respuesta a un problema concreto — no una lista para memorizar.

- Tokenización y normalización
- Stopwords: cuándo ayudan y cuándo destruyen
- Stemming vs. lematización · regex
- Demo en vivo con corpus en español

El pipeline: del texto crudo a términos limpios



“La lluvia provocó inundaciones en Tuxtla”

→ [lluvia, provocar, inundación, tuxtla]

Tokenización: partir no es trivial

Token: la unidad mínima de procesamiento — usualmente una palabra, pero la frontera es una decisión de diseño, no un hecho.

¿Separar por espacios?

“San Cristóbal de las Casas” son 5 tokens... ¿o una sola entidad?

Puntuación y contracciones

“del” = de + el. “U.S.A.”, “3.14”, “\$1,500.00” — el punto no siempre termina oración.

Guiones, emojis, hashtags

“teórico-práctico”, “#Chiapas”, “🙄”: ¿uno, dos o cero tokens?

Idiomas sin espacios

El chino y el japonés no separan palabras — tokenizar es en sí un problema de ML.

Normalización: que lo igual se vea igual

Objetivo: que dos formas superficiales distintas de la misma palabra colapsen en el mismo término.

Operación	Ejemplo	Riesgo en español
Minúsculas (case folding)	Lluvia, LLUVIA → lluvia	“México” (país) vs “méxico”: se pierde la entidad
Quitar acentos	inundación → inundacion	papá → papa, él → el: colapsa palabras distintas
Unicode (NFC/NFKC)	n + ñ → ñ (un codepoint)	casi ninguno — háganlo siempre
Limpiar ruido	URLs, HTML, espacios dobles	a veces la URL o el emoji ERA la señal



Regla práctica: normaliza agresivo para búsqueda, conservador para análisis lingüístico.

Stopwords: filtrar — y cuándo eso destruye información

La idea

Palabras de función (“el”, “de”, “que”, “en”...) aparecen en todos los documentos:
no discriminan, ocupan espacio y dominan los conteos.
Filtrarlas reduce ruido y dimensionalidad.

El peligro

“no me gustó el servicio”

Si “no” es stopword, el documento queda como “gustó servicio” — polaridad invertida.

También: “ser o no ser” queda vacío.

Criterio de decisión

- Búsqueda / topic modeling → filtra: las stopwords solo aportan ruido.
- Análisis de sentimiento → conserva negaciones e intensificadores (“no”, “muy”, “nunca”).
- Nunca uses la lista de NLTK/spaCy a ciegas: ábranla, léanla, éditenla para su dominio.

Stemming vs. lematización

Mismo objetivo — colapsar variantes morfológicas en una forma canónica — con dos filosofías opuestas.

Stemming

Recorta sufijos con reglas heurísticas (Snowball/Porter).
Rápido, sin diccionario.

corriendo → corr

inundaciones → inundacion

El resultado puede no ser una palabra real (“corr”).

Lematización

Usa vocabulario y análisis morfológico para llegar al lema del diccionario (spaCy).

corriendo → correr

fui → ir · mejores → bueno

Más lento, pero lingüísticamente correcto.

Para español: casi siempre lematización. La morfología verbal es tan rica que el stemming produce demasiadas colisiones y fragmentos.

Regex: la herramienta transversal

Antes (y después) de cualquier modelo, las expresiones regulares extraen patrones estructurados escondidos en el texto. Las usarán en cualquier rol de datos, toda su carrera.

```
import re
# correos
r"[\w.+-]+@[\w-]+\.[\w.]+"
# montos en pesos: $1,500.00
r"\$s?\d{1,3}(,\d{3})*(\.\d{2})?"
# fechas dd/mm/aaaa
r"\b\d{1,2}/\d{1,2}/\d{4}\b"
# colapsar espacios
re.sub(r"\s+", " ", texto)
```

Extraer

correos, RFC/CURP, montos, fechas, folios de un corpus.

Limpiar

HTML, URLs, caracteres de control, espacios repetidos.

Validar

que un campo capturado cumple el formato esperado.

Demo + Lab 1 — Pipeline de preprocesamiento



Corpus: noticias chiapanecas (lo reutilizaremos el resto de la unidad). Entorno: notebook + NLTK + spaCy (es_core_news_sm).

1

Cargar y explorar — ¿Cuántos documentos? ¿Longitudes? ¿Qué ruido traen (HTML, URLs, emojis)?

2

Tokenizar y normalizar — Comparar split ingenuo vs. tokenizador real; minúsculas + unicode NFC; decidir qué hacer con acentos.

3

Stopwords con criterio — Aplicar la lista de spaCy, inspeccionarla y modificarla: documenten qué quitaron y por qué.

4

Stemming vs. lematización — Pasar el mismo corpus por ambos; medir vocabulario resultante y elegir uno justificando la decisión.

La decisión del paso 4 la defenderán oralmente: “lo dijo el profesor” no es justificación.

03

Del texto al vector

La idea que conecta esta clase con todo lo que ya saben: si el texto se vuelve vector, toda la Unidad 1 aplica.

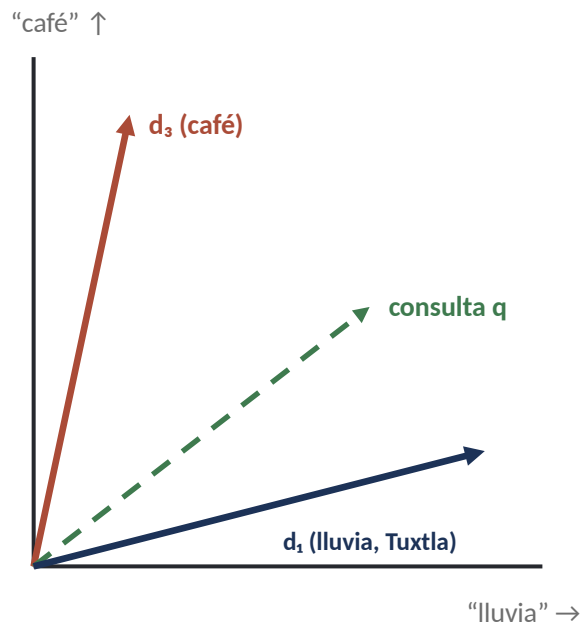
- El modelo de espacio vectorial
- Bag of Words y sus límites
- TF-IDF: la fórmula y su implementación
- Similitud coseno → mini buscador

El modelo de espacio vectorial (Salton, 1960s)

La idea: cada documento es un vector en un espacio donde cada dimensión es un término del vocabulario. La consulta también. Relevancia = cercanía geométrica.

Por qué importa: convierte “buscar texto” en un problema de geometría — y de paso habilita clustering, anomalías y clasificación sobre documentos: toda su Unidad 1.

$d = (w_1, w_2, \dots, w_{|V|})$ — un peso por término del vocabulario



q está “cerca” de $d_1 \rightarrow d_1$ es más relevante

Primera respuesta: Bag of Words

El peso más simple posible: contar cuántas veces aparece cada término. El orden de las palabras se descarta — de ahí “bolsa”.

	lluvia	sequía	chiapas	café	tuxtla	maíz
d_1	1	0	0	0	1	0
d_2	0	1	1	0	0	1
d_3	0	0	1	1	0	0

Cada fila ya es un vector: $d_2 = (0, 1, 1, 0, 0, 1)$. Con vocabularios reales, $|V| \approx 20,000\text{--}100,000$ dimensiones, casi todas en cero (matriz dispersa).

El problema inmediato: en documentos reales, “de”, “la” y “que” tendrían los conteos más altos. El conteo crudo premia lo frecuente, no lo informativo.

La intuición: importancia local $\times$ rareza global

Una palabra es buena descriptora de un documento si cumple dos condiciones a la vez:



Aparece mucho aquí

Si “café” aparece 12 veces en d_3 , ese documento probablemente trata de café.

→ Term Frequency (TF): importancia local.



Aparece poco allá afuera

Si “de” aparece en TODOS los documentos, no distingue nada.

→ Inverse Document Frequency (IDF): rareza global.

La misma lógica de la Unidad 1: lo frecuente define la norma (reglas de asociación); lo raro es lo que informa (anomalías). TF-IDF combina ambas en un solo peso.

Term Frequency: importancia local

$$tf(t, d) = f(t, d) / |d|$$

Frecuencia del término t en el documento d , normalizada por la longitud del documento.

¿Por qué normalizar?

Un documento de 10,000 palabras menciona “café” 12 veces; un tuit lo menciona 2 veces en 15 palabras. Sin normalizar, el largo siempre gana.

Variantes que verán en la literatura

- Conteo crudo: $f(t, d)$ — la base, sensible a longitud.
- Logarítmica: $1 + \log f(t, d)$ — que 100 menciones no valgan $100\times$ más que 1: rendimientos decrecientes.
- Binaria: 1 si aparece, 0 si no — el regreso de Boolean, ahora como caso particular.

Inverse Document Frequency: rareza global

$$\text{idf}(t) = \log(N / \text{df}(t))$$

N = total de documentos; $\text{df}(t)$ = en cuántos documentos aparece t . (Spärck Jones, 1972)

¿Por qué el logaritmo?

Amortigua la escala: con $N = 1\text{M}$ de documentos, una palabra ultra-rara no debe valer un millón de veces más. $\log$ suaviza sin perder el orden.

término	df(t)	idf = $\ln(3/\text{df})$	lectura
en, de, la	3	$\ln(1) = 0.00$	aparece en todo → peso cero
chiapas	2	$\ln(1.5) \approx 0.41$	común en el corpus → peso bajo
sequía	1	$\ln(3) \approx 1.10$	rara → peso alto: discrimina

Con $N = 3$ (nuestro corpus de juguete). Noten: el IDF resuelve matemáticamente lo que las stopwords resolvían a mano.

TF-IDF: el peso que dominó IR por 50 años

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

El peso de un término en un documento es alto solo si es frecuente ahí dentro Y raro allá afuera:

TF alto • IDF alto

“sequía” repetida en d_2

PESO ALTO

TF alto • IDF ≈ 0

“de” repetida en cualquier doc

PESO ≈ 0

TF bajo • IDF alto

mención casual de palabra rara

PESO MODESTO

Cada documento queda como un vector de pesos TF-IDF — listo para medir distancias. Falta una cosa: ¿qué distancia?

Similitud coseno: el ángulo, no la distancia

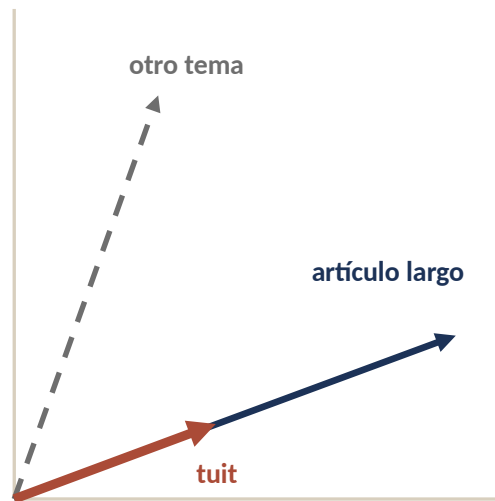
$$\cos(q, d) = (q \cdot d) / (\|q\| \|d\|)$$

1 = misma dirección (máxima similitud) · 0 = ortogonales (nada en común).

¿Por qué no distancia euclidiana?

Un artículo largo sobre lluvia y un tuit sobre lluvia apuntan en la misma dirección, pero el artículo tiene un vector mucho más largo.

Euclidiana los separaría por longitud; el coseno mide solo la orientación temática. En texto, la dirección es el significado.



mismo ángulo $\Rightarrow$ mismo tema, sin importar la longitud

Desde cero: TF-IDF en 15 líneas

```
import numpy as np
from collections import Counter

def tf(doc):
    # importancia local
    c, n = Counter(doc), len(doc)
    return {t: f / n for t, f in c.items()}

def idf(corpus):
    # rareza global
    N = len(corpus)
    df = Counter(t for d in corpus
                 for t in set(d))
    return {t: np.log(N / df[t]) for t in df}

def tfidf(doc, idf_):
    return {t: w * idf_.get(t, 0)
            for t, w in tf(doc).items()}
```

Para notar

- `set(d)` en `df`: contar documentos, no apariciones.
- `idf_.get(t, 0)`: término no visto → peso 0.
- scikit-learn lo hace en una línea (`TfidfVectorizer`) — pero primero se construye, luego se importa.

Lab 2 — Su primer motor de búsqueda



Objetivo: dado el corpus preprocesado del Lab 1, responder consultas en lenguaje libre con un ranking por relevancia. Sin librerías de NLP.

1

Indexar — calcular el vector TF-IDF de cada documento del corpus (con su código del slide anterior).

2

Procesar la consulta — la consulta pasa por el MISMO pipeline de preprocesamiento y se vectoriza con el mismo IDF.

3

Ranquear — similitud coseno entre la consulta y cada documento; devolver el top-5 con sus scores.

4

Romperlo — documenten 2 consultas donde su buscador devuelve resultados malos — y expliquen por qué.

El paso 4 no es opcional: entender dónde falla un modelo vale más que celebrarlo donde funciona.

Busquen “problemas de agua”...

CONSULTA

“problemas de agua”

términos: {problema, agua}

DOCUMENTO RELEVANTE

“La crisis hídrica golpea la región”

términos: {crisis, hídrico, región}

similitud coseno = **0.0** *cero términos compartidos*

TF-IDF trata las palabras como símbolos atómicos sin relación entre sí. No hay peso que arregle que “agua” e “hídrico” sean cadenas distintas. Necesitamos vectores que capturen significado — eso son los embeddings. Semana 7.

Lo que sigue

Próxima sesión

- BM25: la evolución de TF-IDF que Elasticsearch usa en producción en 2026.
- Saturación de TF y normalización de longitud — qué arregla y por qué.
- Comparativa TF-IDF vs. BM25 sobre su propio buscador.

Para antes de la próxima clase

- Terminar Lab 1 + Lab 2 y subirlos a su repo de GitHub Classroom (con AI_USAGE.md).
- Lectura: Aggarwal, cap. 12 (Mining Text Data); Manning et al., caps. 1 y 6.
- Traer sus 2 consultas fallidas del Lab 2: abriremos la clase con ellas.

Hoy construyeron en 3 horas lo que a IR le tomó llegar de 1960 a 1972. La próxima clase: 1994.

Bibliografía de la sesión

-  **Aggarwal, C. C. (2015).** Data Mining: The Textbook. Springer. — Capítulo 12: Mining Text Data.
-  **Manning, C., Raghavan, P. & Schütze, H. (2008).** Introduction to Information Retrieval. Cambridge UP. — Caps. 1, 2 y 6. Gratuito en nlp.stanford.edu/IR-book.
-  **Han, J., Pei, J. & Tong, H. (2023).** Data Mining: Concepts and Techniques (4ª ed.). Morgan Kaufmann. — Texto oficial del curso.
-  **Spärck Jones, K. (1972).** A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 28(1).
-  **Salton, G., Wong, A. & Yang, C. S. (1975).** A vector space model for automatic indexing. Communications of the ACM, 18(11).
-  **Jurafsky, D. & Martin, J. H. (2026).** Speech and Language Processing (3ª ed., borrador). — Caps. 2 y 6, lectura complementaria.